

Data Structures in R

R provides several important data structures for storing and organizing data. The main structures you should teach are:

1. **Vector**
2. **Matrix**
3. **Array**
4. **Factor**
5. **List**
6. **Data Frame**

We also need to understand **indexing**, **grouped data**, and **finding help**.

Vector

A **vector** is the simplest and most commonly used data structure in R.

A vector contains elements of the **same basic data type**.

Creating a numeric vector

```
age <- c(20, 21, 22, 23, 24)
```

age

Output:

```
20 21 22 23 24
```

Character vector

```
name <- c("Rahim", "Karim", "Jannat", "Nadia")
```

Logical vector

```
passed <- c(TRUE, TRUE, FALSE, TRUE)
```

Sequence vector

```
x <- 1:10
```

This creates:

```
1 2 3 4 5 6 7 8 9 10
```

Sequence using seq()

```
x <- seq(1, 20, by = 2)
```

Output:

```
1 3 5 7 9 11 13 15 17 19
```

Repeating values using rep()

```
x <- rep(5, times = 4)
```

Output:

```
5 5 5 5
```

Another example:

```
x <- rep(c("Male", "Female"), times = 3)
```

Output:

```
Male Female Male Female Male Female
```

2. Vector Operations

Suppose:

```
x <- c(10, 20, 30, 40)
```

```
y <- c(2, 4, 6, 8)
```

Addition

```
x + y
```

Subtraction

```
x - y
```

Multiplication

```
x * y
```

Division

```
x / y
```

Other functions

```
sum(x)
```

```
mean(x)
```

```
median(x)
```

```
min(x)
```

```
max(x)
```

```
sd(x)
```

```
length(x)
```

3. Matrix

A **matrix** is a two-dimensional data structure containing elements of the same type.

Creating a Matrix

```
m <- matrix(  
  c(10,20,30,40,50,60),  
  nrow = 2,  
  ncol = 3  
)
```

```
m
```

Output:

```
  [,1] [,2] [,3]  
[1,]  10  30  50  
[2,]  20  40  60
```

Matrix by Row

```
m <- matrix(  
  c(10,20,30,40,50,60),  
  nrow = 2,  
  byrow = TRUE  
)
```

m

Output:

```
  [,1] [,2] [,3]  
[1,]  10  20  30  
[2,]  40  50  60
```

Matrix Functions

nrow(m)

ncol(m)

dim(m)

length(m)

4. Matrix Indexing

Suppose:

```
m <- matrix(  
  1:9,  
  nrow = 3,  
  byrow = TRUE  
)
```

Matrix:

```
  [,1] [,2] [,3]  
[1,]   1   2   3  
[2,]   4   5   6  
[3,]   7   8   9
```

Select element

m[2,3]

Output:

6

Select second row

m[2,]

Select third column

m[,3]

Select a submatrix

m[1:2, 2:3]

5. Array

An **array** is a multidimensional data structure.

Unlike a matrix, an array can have **more than two dimensions**.

Creating a 3-D Array

```
arr <- array(  
  1:24,
```

```
dim = c(3,4,2)
)
```

arr

Here:

- 3 = rows
- 4 = columns
- 2 = layers

Therefore:

$3 \times 4 \times 2 = 24$ elements

Array Indexing

```
arr[1,2,1]
```

This selects:

Row 1, Column 2, Layer 1

You can also select an entire layer:

```
arr[,1]
```

6. Factor

A **factor** is used to represent **categorical data**.

For example:

```
gender <- c("Male","Female","Female","Male","Female")
```

Convert it to a factor:

```
gender_factor <- factor(gender)
```

```
gender_factor
```

Output:

```
Male Female Female Male Female
```

```
Levels: Female Male
```

Find levels

```
levels(gender_factor)
```

Number of levels

```
nlevels(gender_factor)
```

Frequency table

```
table(gender_factor)
```

Example output:

```
Female Male
```

```
3     2
```

Ordered Factor

Suppose students have grades:

```
grade <- c("A", "B", "C", "A", "B")
```

Create an ordered factor:

```
grade <- factor(  
  grade,  
  levels = c("C", "B", "A"),  
  ordered = TRUE  
)
```

Now R knows:

```
C < B < A
```

7. List

A **list** can contain different types of objects.

For example, one list can contain:

- numeric vector
- character vector
- matrix
- data frame

Creating a List

```
student <- list(  
  Name = "Rahim",  
  Age = 20,  
  Marks = c(80, 85, 90),  
  Passed = TRUE  
)
```

```
student
```

This is useful because the elements don't have to be the same type.

Accessing List Elements

Using \$

```
student$Name
```

Output:

```
"Rahim"
```

Using [[]]

```
student[[2]]
```

Output:

```
20
```

Using []

```
student[1]
```

Important difference:

`student[1]`

returns a **list**, whereas:

`student[[1]]`

returns the actual element.

8. Data Frame

A **data frame** is one of the most important structures for statistical analysis.

It is a table consisting of:

- Rows = observations
- Columns = variables

Different columns can have different data types.

Creating a Data Frame

```
student <- data.frame(  
  StudentID = c(101,102,103,104),  
  Name = c("Rahim","Karim","Jannat","Nadia"),  
  Gender = c("Male","Male","Female","Female"),  
  Age = c(20,21,19,22),  
  Marks = c(85,92,78,88)  
)
```

`student`

Output:

StudentID Name Gender Age Marks

101	Rahim	Male	20	85
102	Karim	Male	21	92
103	Jannat	Female	19	78
104	Nadia	Female	22	88

9. Data Frame Functions

Number of rows

`nrow(student)`

Number of columns

`ncol(student)`

Dimensions

`dim(student)`

Variable names

`names(student)`

Structure

```
str(student)
```

Summary

```
summary(student)
```

10. Data Frame Indexing

R uses:

```
data[row, column]
```

For example:

```
student[2,3]
```

means:

Row 2, Column 3

Select second row

```
student[2,]
```

Select third column

```
student[,3]
```

Select rows 1 to 3

```
student[1:3,]
```

Select columns 2 to 4

```
student[,2:4]
```

Select specific rows

```
student[c(1,3),]
```

11. Selecting Columns Using \$

You can select a column using:

```
student$Name
```

or:

```
student$Marks
```

Calculate the average:

```
mean(student$Marks)
```

12. Logical Indexing

This is very important for data analysis.

Suppose we want students whose marks are greater than 80.

```
student[student$Marks > 80,]
```

This returns only students with:

Marks > 80

Multiple Conditions

Students with Marks > 80 **and** Age < 22:

```
student[
  student$Marks > 80 & student$Age < 22,
]
Students from Statistics or Economics:
student[
  student$Department == "Statistics" |
  student$Department == "Economics",
]
```

13. Grouped Data

Grouped data means performing calculations **separately for different groups**.

For example, calculate average marks for each department.

Using dplyr:

```
library(dplyr)
```

```
student %>%
  group_by(Department) %>%
  summarise(
    Mean_Marks = mean(Marks, na.rm = TRUE)
  )
```

Example output:

Department	Mean_Marks
Computer Science	88.5
Economics	86.5
Mathematics	75.0
Statistics	80.0

14. Multiple Grouped Statistics

You can calculate several statistics at once:

```
student %>%
  group_by(Department) %>%
  summarise(
    Number = n(),
    Mean_Marks = mean(Marks, na.rm = TRUE),
    Maximum = max(Marks, na.rm = TRUE),
    Minimum = min(Marks, na.rm = TRUE)
  )
```

This is extremely useful in statistical analysis.

15. Grouping by Gender

```
student %>%  
  group_by(Gender) %>%  
  summarise(  
    Number = n(),  
    Mean_Marks = mean(Marks, na.rm = TRUE)  
  )
```

This answers questions such as:

What is the average mark for male and female students?

16. Grouping by Two Variables

You can group by more than one variable.

```
student %>%  
  group_by(Department, Gender) %>%  
  summarise(  
    Mean_Marks = mean(Marks, na.rm = TRUE),  
    Number = n()  
  )
```

This gives the average marks separately for each **Department** × **Gender** combination.

17. Finding Help in R

R has a built-in help system.

Help for a function

?mean

or:

help(mean)

Search for a function

??regression

Example

?matrix

You can learn:

- Function purpose
- Arguments
- Usage
- Examples
- Return values

18. Important Comparison of R Data Structures

Data Structure Dimensions Same Data Type? Example

Vector	1D	Yes	c(10,20,30)
--------	----	-----	-------------

Data Structure Dimensions Same Data Type? Example

Matrix	2D	Yes	<code>matrix(1:9,3,3)</code>
Array	2D or more	Yes	<code>array(1:24,c(3,4,2))</code>
Factor	1D	Categorical	<code>factor(c("M","F"))</code>
List	Multiple	No	<code>list(Name="Rahim",Age=20)</code>
Data Frame	2D	No	<code>data.frame()</code>

Practical Lab Exercise

Using the following dataset:

```
student <- data.frame(  
  StudentID = c(101,102,103,104,105,106),  
  Name = c("Rahim","Karim","Jannat","Nadia","Sakib","Mim"),  
  Gender = c("Male","Male","Female","Female","Male","Female"),  
  Department = c(  
    "Statistics",  
    "Statistics",  
    "Economics",  
    "Computer Science",  
    "Statistics",  
    "Economics"  
  ),  
  Age = c(20,21,19,22,20,21),  
  Marks = c(85,92,78,88,65,95)  
)
```

Tasks

1. Create a vector containing all students' Marks.
2. Calculate the mean and standard deviation of Marks.
3. Create a matrix containing Age and Marks.
4. Create a factor for Gender.
5. Display the levels of Gender.
6. Create a list containing Name, Age, and Marks.
7. Extract the third element of the list.
8. Display the first three rows of the data frame.
9. Display only the Name and Marks columns.
10. Display students with Marks > 80.

11. Display students from Statistics.
12. Calculate the mean Marks by Department.
13. Calculate the number of students by Gender.
14. Calculate the mean Marks by Department and Gender.